
Assignment 3 Project Documentation

Systems Programming Mini-Project

1 Project Overview

Our project is a distributed machine learning training system built with a parameter-server architecture using TCP sockets. It belongs to **Category 2: Sockets**. One server process stores the current model weights, accepts worker connections, collects gradients, updates the global model, and decides when training should stop. Each worker loads a CSV shard from disk, connects to the server, registers its dataset shape, receives weights, computes a local gradient, and sends it back.

The interaction is command-line based. The user starts the server, which creates a listening socket via `set_up_server_socket()` (`net_utils.c`, lines 21–51) and enters the `select()` loop in `main()` (`server.c`, lines 414–535). Workers are launched with a hostname and shard file; each calls `load_data()` (`data.c`, lines 14–108), `connect_to_server()` (`net_utils.c`, lines 66–103), and `send_register()` (`worker.c`, lines 20–40), then enters its receive loop.

The machine learning component is logistic regression, implemented by `sigmoid()` (`model.c`, line 5), `compute_gradient()` (`model.c`, line 16), `update_weights()` (`model.c`, line 47), and `init_weights()` (`model.c`, line 55). The main systems-programming challenges are reliable socket communication, length-prefixed message framing, partial-read handling, concurrency through `select()`, per-worker state tracking, and robust error handling.

The protocol uses four message types defined in `protocol.h` (lines 7–10): `MSG_REGISTER`, `MSG_WEIGHTS`, `MSG_GRADIENT`, and `MSG_DONE`. Together they implement a full training workflow: register, distribute weights, collect gradients, and broadcast the final model.

2 Build Instructions

The Makefile defines:

- `PORT = 4242` (Makefile, line 1)
- `FLAGS = -Wall -Wextra -g -DPORT=$(PORT)` (Makefile, line 2)

The default build target is:

```
all: server worker gen_data
```

as defined in Makefile (line 4). Therefore, running `make` produces three executables:

- `server`
- `worker`
- `gen_data`

Build

```
make
```

Generate test shards

```
./gen_data <num_features> <total_samples> <num_shards>
```

For example, `./gen_data 3 1000 4` generates `shard_0.csv` through `shard_3.csv`. The generator is implemented in `main()` (`gen_data.c`, lines 13–79).

Run the server

```
./server  
./server 3
```

The optional argument specifies the expected number of workers (default 1). Argument parsing is in `main()` (`server.c`, lines 381–384).

Run workers

```
./worker <server-host> <shard_file>
```

For example: `./worker localhost shard_0.csv`. The worker parses these arguments in `main()` (`worker.c`, lines 190–197).

Input files

Each shard is a CSV file where each line has the form `feature1,feature2,...,featureN,label`. The final column is the binary label and the preceding columns are floating-point features. The shard-loading logic is in `load_data()` (`data.c`, lines 14–108).

Cleanup

```
make clean
```

The cleanup rule is defined in `Makefile` (lines 36–37) and removes `*.o`, `server`, `worker`, and `gen_data`.

3 Architecture Diagram

Figure 1 shows the overall structure. The server accepts connections via `accept_connection()` (`net_utils.c`, lines 54–63) and monitors them in its `select()` loop (`server.c`, lines 414–535). Each worker connects via `connect_to_server()` (`net_utils.c`, lines 66–103) and enters its own receive loop (`worker.c`, lines 239–301). Worker-to-server traffic consists of registration and gradient messages; server-to-worker traffic consists of weight broadcasts and the termination message.

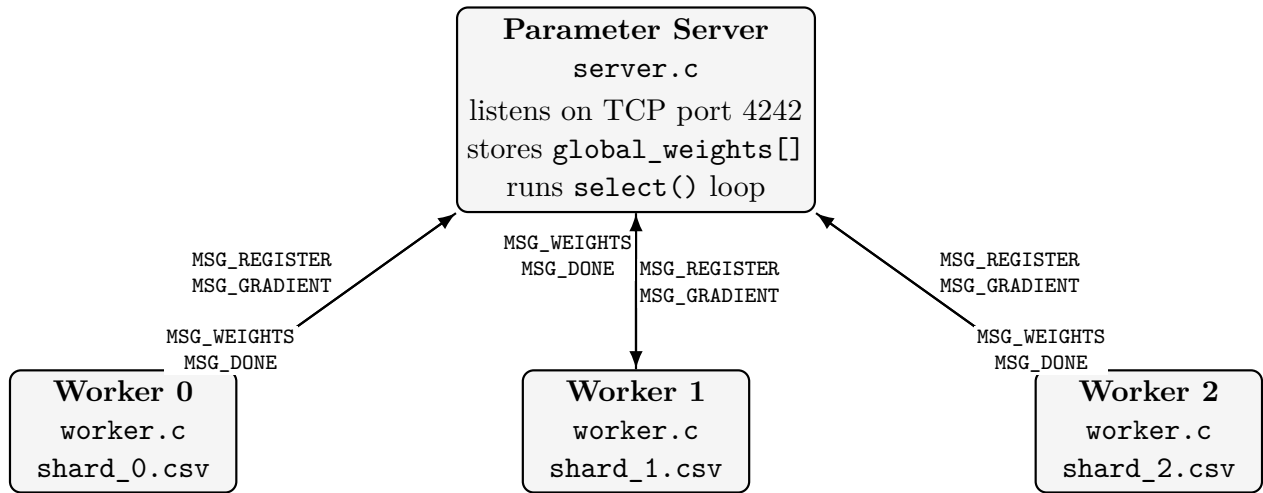


Figure 1: Overall system architecture.

4 Communication Protocol

4.1 Common Framing and Header Format

All messages share a common wire format: a 5-byte header followed by a variable-length payload. The header consists of a 1-byte message type code at offset 0, followed by a 4-byte payload size in network byte order (big-endian) at offsets 1–4. Multi-byte integers in the payload are also in network byte order, converted with `htonl/ntohl`. Floats are transmitted in host byte order; since both server and workers run on teach.cs (x86-64), no float byte-swapping is needed.

A shared helper function in `io_utils` handles header serialization: `send_header(int fd, uint8_t type, uint32_t payload_len)` constructs a 5-byte buffer. Byte 0 is the type code, and bytes 1–4 are `htonl(payload_len)`. It sends the buffer via `write_all()`. Returns 0 on success, `-1` on failure. (`io_utils.c`, line 77) On the receiving side, header parsing is done inline: the receiver reads byte 0 as the type code, copies bytes 1–4 into a `uint32_t`, and converts with `ntohl` to obtain the host-order payload length.

Header byte layout (all messages)

Offset	Size (bytes)	Field	Encoding
0	1	type	Message type code (0x01–0x04)
1	4	payload_len	<code>htonl(payload_len)</code> ; payload size in bytes

On the worker side, complete blocking reads are handled by `read_all()` (`io_utils.c`, lines 13–23). On the server side, partial nonblocking message assembly is handled by `accumulate_read()` (`io_utils.c`, lines 48–71). This structure avoids ambiguity in message boundaries and allows the server to handle fragmented TCP input safely.

4.2 MSG_REGISTER (0x01)

Direction: Worker $\rightarrow$ Server, sent once immediately after connecting. **Total message size:** $5 + 8 = 13$ bytes.

Byte layout

Offset	Size	Field	Encoding
0	1	type	0x01 (MSG_REGISTER)
1	4	payload_len	htonl(8)
5	4	num_features	htonl(num_features)
9	4	num_samples	htonl(num_samples)

Semantics. `send_register()` (`worker.c`, lines 20–40) serializes both fields with `htonl()` and sends the message with `write_all()`.

On the server side, `dispatch_message()` (`server.c`, lines 337–369) routes the message to `handle_register()` (`server.c`, lines 287–327), which deserializes and validates the feature and sample counts, stores them in the `worker_info` slot, and sets the worker’s state to registered (state 1). If this is the first worker, the server also initializes the global weights with `init_weights()` (`model.c`, lines 55–60). The server does not broadcast weights immediately; training starts later in `main()` (`server.c`, lines 485–501) once enough workers have registered.

Error handling

- If the worker sends an invalid feature count, `handle_register()` (`server.c`, lines 300–304) logs an error and disconnects the worker through `handle_disconnect()` (`server.c`, lines 278–285).
- If the worker sends a nonpositive sample count, `handle_register()` (`server.c`, lines 316–320) logs an error and disconnects the worker.
- If a later worker’s feature count does not match the server’s established feature count, `handle_register()` (`server.c`, lines 306–314) disconnects that worker.

4.3 MSG_WEIGHTS (0x02)

Direction: Server → Worker, sent at the start of each round (first broadcast after enough workers register).

Total message size: $5 + 8 + n \times 4$ bytes, where n is the number of features.

Byte layout

Offset	Size	Field	Encoding
0	1	type	0x02 (MSG_WEIGHTS)
1	4	payload_len	htonl(8 + $n \times \text{sizeof(float)}$)
5	4	round	htonl(round)
9	4	num_features	htonl(n)
13	$n \times 4$	weights	float[n], host byte order

Semantics

`send_weights_to_worker()` (`server.c`, lines 46–78) constructs the message in a stack buffer and sends it with `write_all()`, advancing the worker’s state and resetting `gradient_received`. The server broadcasts to all registered workers through `broadcast_weights()` (`server.c`, lines 85–95). On the worker side, parsing is inline in `main()` (`worker.c`, lines 250–285): it reads the header with `read_all()`, validates the payload size, reads the payload, deserializes round and feature count with `ntohl`, and copies the weights into local memory.

Error handling

- If the payload size is too small (fewer than 8 bytes), the worker logs an error, drains the remaining payload bytes, and continues to the next message (`worker.c`, lines 251–258). If the payload exceeds the local buffer size, the worker logs an error and exits the receive loop (`worker.c`, lines 260–263).
- If the received feature count exceeds the shard’s feature count, the worker truncates to the shard’s value (`worker.c`, line 272).
- If a broadcast send fails on the server side, `broadcast_weights()` disconnects the affected worker through `handle_disconnect()` (`server.c`, lines 278–285).

4.4 MSG_GRADIENT (0x03)

Direction: Worker → Server, sent once per round after computing the local gradient. **Total message size:** $5 + 12 + n \times 4$ bytes.

Byte layout

Offset	Size	Field	Encoding
0	1	type	0x03 (MSG_GRADIENT)
1	4	payload_len	htonl(12 + n * sizeof(float))
5	4	round	htonl(round); current training round
9	4	num_features	htonl(n)
13	4	local_loss	float, host byte order; average loss on this worker’s shard
17	$n \times 4$	gradients	float[n], host byte order; gradient vector

Semantics

Once `accumulate_read()` assembles a complete message, `dispatch_message()` (`server.c`, line 337) routes it to `handle_gradient()` (`server.c`, line 123), which deserializes each field with `memcpy/ntohl`, validates `num_features` against `MAX_FEATURES` (lines 138–142) and `round` against `current_round` (line 150), then stores the gradient and sets `gradient_received = 1` (lines 157–159). On validation failure it returns `-1`; the caller logs a warning but does not disconnect the worker. The gradient barrier and aggregation logic are described in Section 5.

On the worker side, `send_gradient()` (`worker.c`, line 116) serializes the payload with `memcpy/htonl`, then sends header and payload via `send_header()` and `write_all()` (lines 140–141).

Error handling

- If `num_features` is out of range (zero, negative, or exceeds `MAX_FEATURES`), `handle_gradient()` prints a diagnostic to `stderr` and returns `-1` without copying any gradient data. The worker’s `gradient_received` remains 0, so the malformed gradient is not counted toward aggregation. This prevents a buffer overflow on `w->gradients`.
- If `round` does not match `current_round`, the gradient is stale. The server logs a warning to `stderr` ("Stale gradient: got round %d, expected %d") and returns `-1` without modifying any state. This can happen if a worker was slow and is still sending a gradient from a previous round’s weights.
- If `write_all()` fails on the worker side (returns `-1`), `send_gradient()` prints an error to `stderr` and returns `-1`. The caller treats this as a server disconnect, closes the socket, and exits.

- If the worker disconnects before sending its gradient, `accumulate_read()` returns `-1` and the server cleans up the worker via `handle_disconnect()`. The remaining active workers' gradients are still collected normally.

4.5 MSG_DONE (0x04)

Direction: Server $\rightarrow$ all active workers, sent exactly once when `check_termination()` (`server.c`, line 229) returns 1 (global loss below `LOSS_THRESHOLD` (0.001) or `current_round` reaches `MAX_ROUNDS` (100)). **Total message size:** $5 + 8 + n \times 4$ bytes.

Byte layout

Offset	Size	Field	Encoding
0	1	type	0x04 (MSG_DONE)
1	4	payload_len	htonl(8 + n * sizeof(float))
5	4	num_features	htonl(n)
9	4	final_loss	float, host byte order; the last global loss value
13	$n \times 4$	weights	float[n], host byte order; the final trained model

Semantics

On the server side, `broadcast_done()` (`server.c`, line 241) builds the payload once, then for each active worker (`fd != -1`, `state == 2`) sends the header via `send_header()` and the payload via `write_all()` (lines 262–263). On the worker side, when the main receive loop (`worker.c`, line 239) reads a header with type `MSG_DONE`, `handle_done()` (`worker.c`, line 153) reads the payload with `read_all()` (line 159), deserializes all fields with `memcpy/ntohl`, prints the final loss and weights to stdout (lines 178–183), and returns 1. The main loop then breaks and the worker cleans up.

Error handling

- If `write_all()` fails for a particular worker during broadcast, the server logs an error to stderr and continues sending to the remaining workers. The function returns `-1` to indicate that at least one send failed.
- If `read_all()` fails on the worker side (server disconnected before finishing the message), `handle_done()` prints an error to stderr and returns `-1`. The main loop breaks regardless and the worker performs its normal cleanup (closing the socket and freeing memory).

5 Concurrency Model

This project uses Category 2 concurrency: a single-threaded server manages multiple worker clients through the `select()` system call. There are no threads and no `fork()`-per-client.

5.1 The `select()` Loop

The server's main loop (`server.c`, line 414) follows the `simpleselect.c` pattern. Two `fd_set` variables are maintained: `allset` tracks the listening socket plus every connected worker, and `rset` is a copy passed to `select()`. Each iteration copies `rset = allset` and resets a 10-second timeout (lines 415–419), then calls `select(maxfd + 1, &rset, NULL, NULL, &tv)`. On timeout the server logs a status message and continues; on `EINTR` it retries (line 423).

After `select()` returns with readable descriptors, the server checks two things in order:

1. **New connections.** If `FD_ISSET(listen_fd, &rset)` is true (line 433), the server accepts the connection with `accept_connection()`, adds the new fd to `allset` with `FD_SET()` (line 441), updates `maxfd` if needed (line 442), and initializes a `worker_info` slot via `add_worker()`.
2. **Existing workers.** The server iterates over all `MAX_WORKERS` slots (line 448). For each slot with a valid fd that is set in `rset`, it calls `accumulate_read()` (line 450).

5.2 How Blocking on a Single Client Is Avoided

The server never calls a blocking read loop for any individual client. `accumulate_read()` (`io_utils.c`, line 48) performs exactly one `read()` call (line 49), appending whatever bytes are available to that worker's `recv_buf`. It returns 0 if the header or payload is still incomplete (lines 57, 67), or 1 when a full message is ready (line 70). Because only one system call is made per worker per iteration, a slow worker does not block service to others.

When `accumulate_read()` returns 1, the server loops (line 470) calling `dispatch_message()` (line 337) to route each complete message to `handle_register()` or `handle_gradient()` by type byte. After processing, `dispatch_message()` shifts remaining bytes forward with `memmove()` (lines 364–368), handling the case where a single `read()` delivered multiple messages.

When `accumulate_read()` returns `-1` (disconnect or error), the server calls `handle_disconnect()` (`server.c`, line 278) to close the fd and reset the `worker_info` slot. The calling code in the main loop then removes the fd from `allset` with `FD_CLR()` (line 457) and recalculates `maxfd` by scanning all active workers if the disconnected fd was the current `maxfd` (lines 459–466).

5.3 Synchronous Gradient Barrier

At the end of each select loop iteration, `all_gradients_received()` (`server.c`, line 167) checks whether every active worker (`fd != -1`, `state == 2`) has `gradient_received == 1`. It guards against vacuous truth: if `count_active_workers()` (line 107) returns 0, the function returns 0. When all gradients are in, `aggregate_and_update()` (line 187) computes a weighted average and updates the model. If `check_termination()` (line 229) returns true, `broadcast_done()` sends the final model; otherwise `broadcast_weights()` starts the next round (lines 516–532). This barrier ensures no worker receives new weights until all active workers have reported, without any blocking beyond the select call itself.

Workers use blocking I/O (`read_all()` in `io_utils.c`, line 13; `write_all()` in `io_utils.c`, line 29) because each worker communicates with only one server. Both functions loop until the requested number of bytes has been transferred or an error occurs, so the worker never tries to parse partial headers or payloads.

6 Error Handling and Robustness

Both the server and worker call `signal(SIGPIPE, SIG_IGN)` at startup so that writes to a closed socket produce an EPIPE error (caught via the `write()` return value) instead of terminating the process. On the server side, this call is at `server.c` line 401. On the worker side, the call is at `worker.c` line 204, in `main()` before any I/O begins.

6.1 Bad Behavior 1: Worker Disconnects Mid-Training

Detection

When the server calls `accumulate_read()` for a worker whose connection has been closed, `read()` returns 0 (EOF) or `-1` (error). `accumulate_read()` returns `-1` in either case (`io_utils.c`, line 51).

Handling

The server calls `handle_disconnect()` (`server.c`, line 278) to close the `fd` and reset the `worker_info` slot. The main loop then removes the `fd` from `allset` with `FD_CLR()` (line 457) and recalculates `maxfd` (lines 459–466). Since `all_gradients_received()` and `aggregate_and_update()` only consider workers with `fd != -1`, the disconnected worker is excluded from both the gradient barrier and the weighted average. A vacuous-true guard in `all_gradients_received()` (line 169) prevents spurious aggregation when zero workers remain. A separate check at line 503 exits the server gracefully when all workers have disconnected.

6.2 Bad Behavior 2: Partial Read on Gradient Message

Detection

TCP does not guarantee that a full application-level message arrives in a single `read()` call. When the server calls `accumulate_read()` and `read()` returns fewer bytes than needed for a complete `MSG_GRADIENT` message, the function detects this by comparing `recv_len` against `HEADER_SIZE + payload_size` (`io_utils.c`, line 67).

Handling

`accumulate_read()` returns 0, leaving the partial data in the worker’s `recv_buf`. The server returns to the select loop and continues servicing other workers normally. The next time this worker’s `fd` becomes readable, `accumulate_read()` appends more bytes to `recv_buf` at the current `recv_len` offset (line 49). This repeats until the full message (header + payload) is assembled, at which point `accumulate_read()` returns 1 and the message is dispatched. The per-worker `recv_buf` and `recv_len` fields in `struct worker_info` (`protocol.h`, line 31) make this possible. Each worker’s partial state is tracked independently.

6.3 Bad Behavior 3: Worker Sends Gradient for Wrong Round (Stale Gradient)

Detection

Each `MSG_GRADIENT` message includes a `round` field. When `handle_gradient()` (`server.c`, line 123) deserializes this field (line 129), it compares the received round against the server’s `current_round` (line 150).

Handling

If the rounds do not match, the gradient is stale, likely from a slow worker that was still computing from a previous round’s weights. The server prints a warning to `stderr` ("`Stale gradient: got round %d, expected %d`") and returns `-1` without modifying any state. The gradient is not counted toward the current round’s aggregation and `gradient_received` remains 0 for that worker. The server will wait for this worker to send a gradient with the correct round number before proceeding.

7 Team Contributions

Angela Su was primarily responsible for the gradient-aggregation and training-loop side of the project. This included implementing the logistic-regression model functions in `model.c`, the worker-to-server `MSG_GRADIENT` path, the server-side gradient handling and aggregation logic, the training termination condition, and the final `MSG_DONE` broadcast.

Jinxi Li was primarily responsible for the connection-management and weight-broadcast side of the project, together with the networking and data-loading foundation. This included implementing the TCP helper functions in `net_utils.c`, the CSV shard loader in `data.c`, the worker-side registration path, and the server-side worker admission, registration handling, and weight broadcast logic.